Billionaire Brother

Agentic MCP REST API Specification

Document Version: 1.0.0

Status: Deployed & Active

Base URL: https://thebillionairebrother.com

Auth Method: Bearer Token (Authorization Header)

Target Audience: AI Agents, Workflows, External Automations

This document outlines the machine-consumable endpoints developed to allow AI agents to securely inspect, update, and manage workspace database tables directly, bypassing standard row-level security (RLS) via a pre-shared API key.

1. System Architecture Overview

The Billionaire Brother Agentic MCP (Machine-Consumable Platform) REST API layer provides a secure, stateless interface for AI agents and scripts to interact directly with the backend database. Unlike standard client-facing endpoints which use Row-Level Security (RLS) and OAuth sessions, the MCP layer bypasses RLS using a service-role admin client. It enforces authentication via a pre-shared API key and strictly whitelists operations to prevent unauthorized database modifications.

Authentication & Security Details

Every request to the MCP endpoints must present the header:
Authorization: Bearer <MCP_API_KEY>
If the header is missing, incorrect, or the key is not configured on the server, the endpoint returns a 401 Unauthorized or 500 Server Error.

Response Format

All endpoints return a consistent JSON response schema:
• **Success (HTTP 200/201):** { "success": true, "data": ... }
• **Failure (HTTP 4xx/5xx):** { "success": false, "error": "Human-readable error description" }

Routing Conventions

The endpoints follow a standard dynamic layout:
• GET /api/mcp/[resource] — Lists records with support for filtering, sorting, and limits.
• POST /api/mcp/[resource] — Creates a new record. Whitelists inputs and validates required fields.
• GET /api/mcp/[resource]/[id] — Retrieves details of a specific record by primary key.
• PATCH /api/mcp/[resource]/[id] — Updates allowed fields of a specific record by ID.
• DELETE /api/mcp/[resource]/[id] — Deletes a specific record by ID.

2. Exposed Resources Reference Table

The following table details the schemas exposed via the MCP API. For each resource, specific whitelisted fields, required fields, and parent filter rules are configured in the system.

Resource	Parent Filter Field	Required Fields	Allowed Whitelist Fields
users	`None`	email	email, display_name, stripe_customer_id, subscription_status, subscription_plan, onboarding_complete
business_profiles	`user_id`	user_id	business_name, business_state, industry, current_revenue_range, strengths, weaknesses, risk_tolerance, hours_per_week, monthly_budget_range, no_go_constraints, target_audience, existing_assets, additional_context, raw_answers
founder_profiles	`user_id`	user_id	team_size, va_count, calendar_blocks_available, timezone
decisions	`user_id`	user_id	business_profile_id, status, chosen_strategy_id, thread_id, chosen_at
strategy_options	`decision_id`	decision_id, rank, archetype, thesis	rank, archetype, thesis, channel_focus, offer_shape, first_7_day_plan, risks, mitigations, kpis, decision_score, confidence, score_breakdown, assumptions, raw_ai_output
execution_contracts	`user_id`	user_id, decision_id, strategy_id, locked_kpi, weekly_deliverable, calendar_blocks	decision_id, strategy_id, locked_kpi, weekly_deliverable, calendar_blocks
weekly_cycles	`user_id`	user_id, week_number	execution_contract_id, week_number, status, kpi_target, kpi_actual, board_meeting_notes, kill_list, keep_list, double_list, thread_id, completed_at
deliverables	`weekly_cycle_id`	user_id, weekly_cycle_id, department, title, size	department, title, size, status, content, red_team_passed, red_team_feedback
tasks	`weekly_cycle_id`	user_id, weekly_cycle_id, title	deliverable_id, title, description, assignee, status, due_date, sort_order
assets	`user_id`	user_id, title, asset_type	deliverable_id, asset_type, title, content, storage_path
assumptions	`strategy_option_id, weekly_cycle_id`	assumption_text	strategy_option_id, weekly_cycle_id, assumption_text, category, risk_level
generation_jobs	`user_id`	user_id, job_type	job_type, reference_id, status, attempts, max_attempts, error_message, started_at, completed_at
chat_conversations	`user_id`	user_id, title	execution_contract_id, title
chat_messages	`conversation_id`	conversation_id, role, content	role, content, reaction, task_updates

3. Sample Requests & Responses

Below are realistic examples of how an AI agent or developer can interact with the Billionaire Brother MCP REST API. For all examples, the base URL is: **https://thebillionairebrother.com**

Example 1: List Users (GET)

Retrieve users matching a specific filter (e.g. email query).

```
curl -X GET "https://thebillionairebrother.com/api/mcp/users?email=founder@example.com" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY"
```


```
{
  "success": true,
  "data": [
    {
      "id": "usr_8f3c7a2b-81d3-41c6",
      "email": "founder@example.com",
      "display_name": "John Doe",
      "stripe_customer_id": "cus_P92k8sL2",
      "subscription_status": "active",
      "subscription_plan": "scale",
      "onboarding_complete": true,
      "created_at": "2026-07-01T10:15:30Z"
    }
  ]
}
```


Example 2: Create Business Profile (POST)

Creates a new profile. Requires user_id, and only whitelisted fields are stored.

```
curl -X POST "https://thebillionairebrother.com/api/mcp/business_profiles" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": "usr_8f3c7a2b-81d3-41c6",
    "business_name": "Apex Agency",
    "business_state": "California",
    "industry": "B2B SaaS Marketing",
    "current_revenue_range": "$10k-$50k/mo",
    "strengths": ["Outbound Sales", "Copywriting"],
    "weaknesses": ["Technical SEO", "Paid Ads"],
    "risk_tolerance": "Medium"
  }'
```


```
{
  "success": true,
  "data": {
    "id": "prof_03da8b21-42e1-4560",
    "user_id": "usr_8f3c7a2b-81d3-41c6",
    "business_name": "Apex Agency",
    "business_state": "California",
    "industry": "B2B SaaS Marketing",
    "current_revenue_range": "$10k-$50k/mo",
    "strengths": ["Outbound Sales", "Copywriting"],
    "weaknesses": ["Technical SEO", "Paid Ads"],
    "risk_tolerance": "Medium",
    "created_at": "2026-07-04T02:00:00Z"
  }
```

```
}
```

Example 3: Update Task (PATCH)

Updates only specific whitelisted fields on a task, such as the status or assignee.

```
curl -X PATCH "https://thebillionairebrother.com/api/mcp/tasks/tsk_99a8b7c6-3d2e-4f1a" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "status": "completed",
    "assignee": "John Doe"
  }'
```

```
{
  "success": true,
  "data": {
    "id": "tsk_99a8b7c6-3d2e-4f1a",
    "deliverable_id": "del_d1f2e3a4-56b7-89c0",
    "weekly_cycle_id": "cyc_w1e2c3d4-5678-90ab",
    "user_id": "usr_8f3c7a2b-81d3-41c6",
    "title": "Implement Email Sequences",
    "description": "Write copy and trigger in CRM",
    "assignee": "John Doe",
    "status": "completed",
    "due_date": "2026-07-10",
    "sort_order": 10,
    "updated_at": "2026-07-04T02:30:15Z"
  }
}
```

Example 4: List Chat Messages with Parent Filter (GET)

Get messages belonging to a specific conversation. Note that `chat_messages` requires the `conversation_id` query parameter.

```
curl -X GET "https://thebillionairebrother.com/api/mcp/chat_messages?conversation_id=conv_c3f4a5b6" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY"
```

```
{
  "success": true,
  "data": [
    {
      "id": "msg_7f8a9b0c",
      "conversation_id": "conv_c3f4a5b6",
      "role": "user",
      "content": "Generate my strategy options for this week.",
      "created_at": "2026-07-03T18:00:00Z"
    },
    {
      "id": "msg_1a2b3c4d",
      "conversation_id": "conv_c3f4a5b6",
      "role": "assistant",
      "content": "Starting the weekly generation job. I will check for insights.",
      "created_at": "2026-07-03T18:00:05Z"
    }
  ]
}
```

Example 5: Chat with Billionaire Brother (POST)

Send a message to Derek (the Billionaire Brother) on behalf of a specific user. This loads the user's business profile and tasks, interacts with Gemini, and returns the response.

```
curl -X POST "https://thebillionairebrother.com/api/mcp/chat" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": "usr_8f3c7a2b-81d3-41c6",
    "message": "Yo Derek, give me a quick 1-sentence tip on outbound marketing."
  }'
```


```
{
  "success": true,
  "data": {
    "response": "Focus on personalized value-first offers; don't pitch your services, pitch a specific
 quick-win solution they can't say no to.",
    "reaction": "let's go",
    "gifUrl": "https://media.giphy.com/media/.../giphy.gif"
  }
}
```

4. Deployment & Verification

To verify or connect new integrations, follow these steps:

Step 1: Check Environment Variables
Ensure the server-side environment contains MCP_API_KEY and SUPABASE_SERVICE_ROLE_KEY. These are loaded dynamically in Next.js backend and are never exposed to the client bundle.

Step 2: Dry Run Test (Ping Endpoint)
Run the following curl command in your terminal to verify that the authentication layer correctly responds. A healthy response will show either successful data retrieval or resource filtering error messages (if a parent filter is missing), while an incorrect token will result in a 401 Unauthorized error.

```
curl -i -X GET "https://thebillionairebrother.com/api/mcp/users" \
  -H "Authorization: Bearer YOUR_MCP_API_KEY"
```

End of Document.